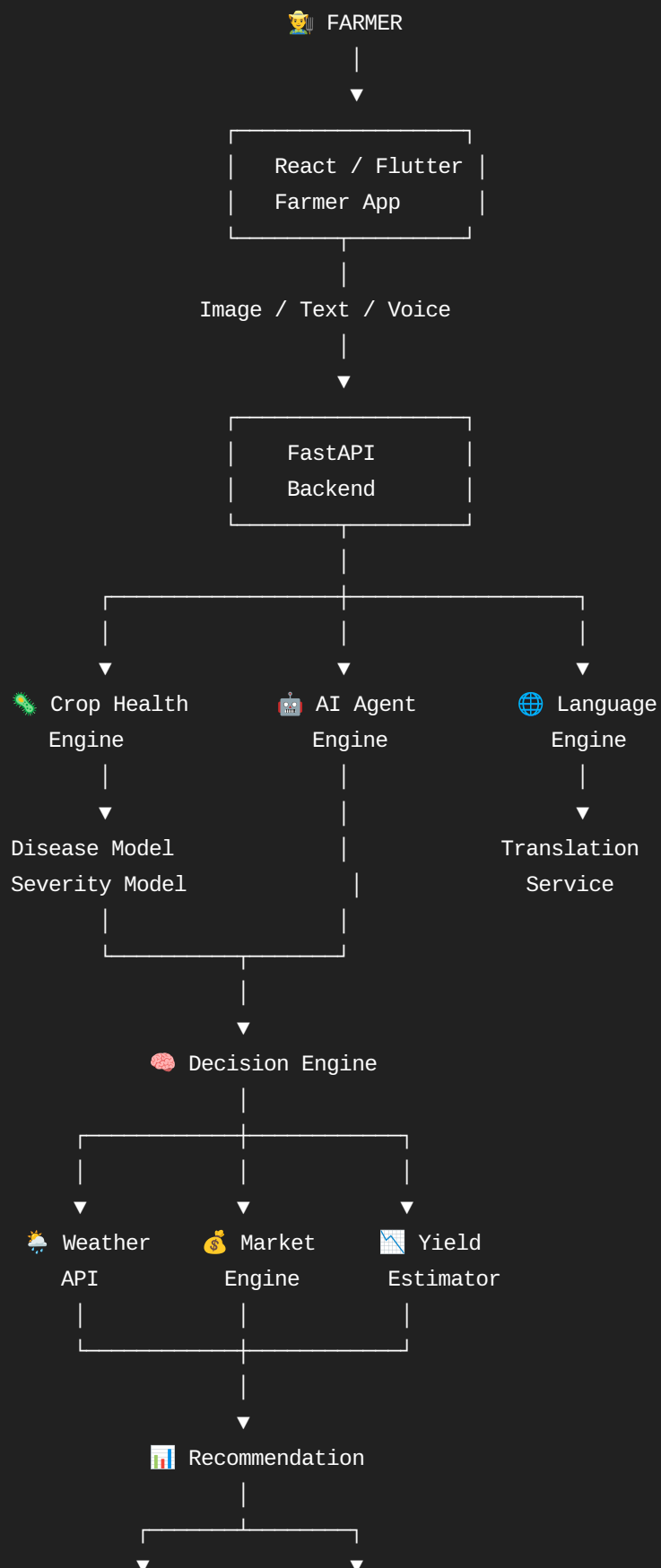
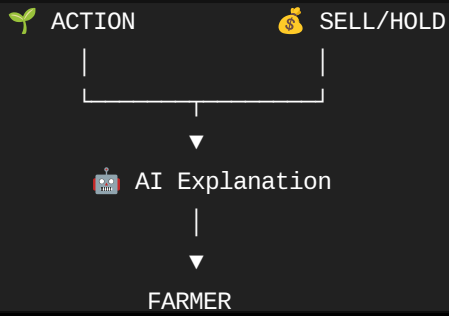


Absolutely. Let's build the architecture around what you're **actually going to implement**, not the entire wish-list.

AI Farm Companion — System Architecture

The system should follow this overall pipeline:





1. 🧑🌾 Farmer Interface

We'll keep the frontend simple.

Main screens

HOME

- 📷 Diagnose Crop
- 💬 Ask AI
- 💰 Market
- 🌤️ Weather
- 🧑🌾 Community
- 🌐 Language

The farmer shouldn't have to understand what's happening behind the scenes.

2. 💻 Backend — FastAPI

This is the **central controller**.

Everything goes through the backend.

React
↓
FastAPI
↓
Services

I'd structure the backend approximately like:

```
backend/
|
├─ main.py
|
├─ routes/
|   ├── crop.py
|   ├── chat.py
|   ├── market.py
|   ├── weather.py
|   ├── recommendation.py
|   ├── community.py
|   └─ translation.py
|
├─ services/
|   ├── disease_service.py
|   ├── llm_service.py
|   ├── weather_service.py
|   ├── market_service.py
|   ├── prediction_service.py
|   ├── yield_service.py
|   └─ decision_service.py
|
├─ models/
|   ├── disease_model/
|   └─ price_model/
|
├─ database/
|
└─ utils/
```

This separation will make your project much easier to develop as a team.

3. Crop Health Engine

This is your computer vision component.

Input

Crop image



Model

Hugging Face / TensorFlow model



Output

JSON

```
{  
  "crop": "Tomato",  
  "disease": "Early Blight",  
  "confidence": 0.94  
}
```

But I recommend going one step further.

Your model should eventually provide:

Crop
Disease
Confidence
Severity

For example:

Tomato
Early Blight
94% confidence
Moderate severity

4. AI Agent

This is the **brain that communicates with the farmer**.

The LLM should **NOT independently decide everything**.

Instead, it receives structured information from your other services.

For example:

JSON

```
{  
  "crop": "Tomato",  
  "disease": "Early Blight",  
  "confidence": 0.94,  
  "severity": "Moderate",  
  "weather": {  
    "rain_probability": 80,  
    "humidity": 87  
  }  
}
```

```
{  
  },  
  "market": {  
    "current_price": 24,  
    "predicted_price_3d": 28  
  },  
  "estimated_yield_loss": 18  
}
```

Then the LLM converts this into **human-friendly advice**.

5. 🌤️ Weather Service

Your backend calls a weather API.

Input:

Farmer location

Output:

Temperature
Humidity
Rain probability
Rainfall
Wind
Forecast

We don't want the LLM guessing weather.

Instead:

```
Weather API  
  ↓  
FastAPI  
  ↓  
Structured weather data  
  ↓  
AI Agent
```

6. 💰 Market Intelligence Engine

This has two parts.

Current Market Price

Get historical/current market data.

Example:

Tomato

Yesterday → ₹22/kg

Today → ₹24/kg

Price Prediction

Your ML model uses historical prices.

Something like:

Historical Data

↓

Data Cleaning

↓

Feature Engineering

↓

Regression / Time-Series Model

↓

Future Price

For the hackathon, **don't overcomplicate this with an enormous forecasting system.**

A well-explained baseline model is better than a fancy model you can't justify.

7. Yield-Loss Estimator

This takes:

Disease

+

Severity

+

Affected Area

+

Crop

and produces:

Estimated yield loss = 15-20%

Then you can convert that into money.

For example:

Expected production = 2,000 kg

Estimated loss = 18%

Potential loss $\approx$ 360 kg

Current price = ₹24/kg

Potential revenue impact $\approx$ ₹8,640

This makes your system much more meaningful to a farmer.

8. 🧠 Decision Engine

This is arguably the most important component.

Don't let the LLM randomly decide:

"I think you should sell."

Instead, create a **deterministic decision layer**.

It takes:

Crop Health
+
Disease Severity
+
Weather Risk
+
Current Price
+
Predicted Price
+
Yield Loss

↓

Decision Engine

↓

SELL
HOLD
or
SELL PARTIALLY

I would actually recommend adding "**Sell Partially**".

For example:

Recommendation: Sell 40%, Hold 60%.

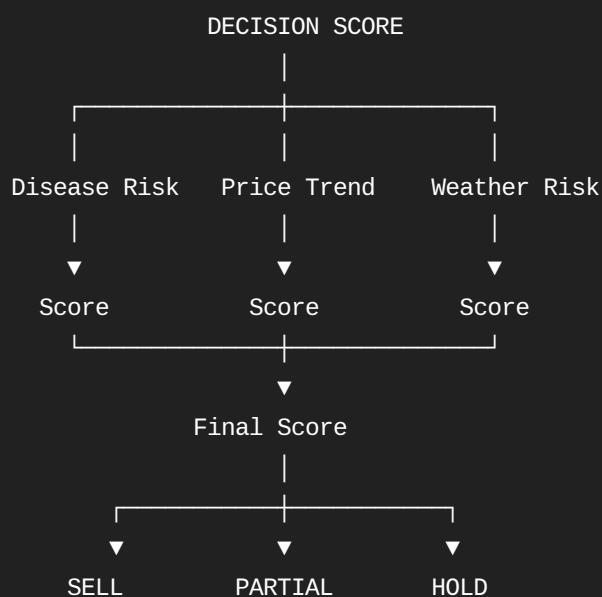
That's much more realistic than forcing every situation into binary Sell/Hold.

The LLM then explains the decision.

9. Decision Logic

For the hackathon, we can create a scoring system.

For example:



Later we'll define the **actual weights and formula**.

10. Multilingual Layer

This should sit between the farmer and AI.


```
graph TD; Farmer --> LD[Language Detection]; LD --> T1[Translation]; T1 --> AI[AI Agent]; AI --> R[Response]; R --> T2[Translation]; T2 --> Farmer
```

Example:

```
graph TD; Kannada --> EIR[English/internal representation]; EIR --> AIreasoning[AI reasoning]; AIreasoning --> KannadaResponse[Kannada response]
```

The internal system can remain primarily English while the farmer interacts in their preferred language.

11. 🧑🌾 Community Q&A

This can be relatively independent from the core agricultural engine.

```
graph TD; Farmer --> PQ[Post Question]; PQ --> DB[(Database)]; DB --> Community; Community --> Answers
```

And optionally:

```
graph TD; Question --> LLM
```



AI Suggested Answer

You could show:

AI Suggested Answer

and then:

Community Answers

This is a nice differentiator.

12. Database

For the hackathon, I'd use **PostgreSQL**.

Potential tables:

USERS

user_id
name
phone
location
preferred_language

CROP_DIAGNOSES

diagnosis_id
user_id
crop
disease
confidence
severity
image_url
created_at

MARKET_PRICES

crop
market
date
price

WEATHER

location
date
temperature
humidity
rainfall

RECOMMENDATIONS

recommendation_id
user_id
crop
decision
reason
created_at

COMMUNITY_POSTS

post_id
user_id
question
crop
created_at

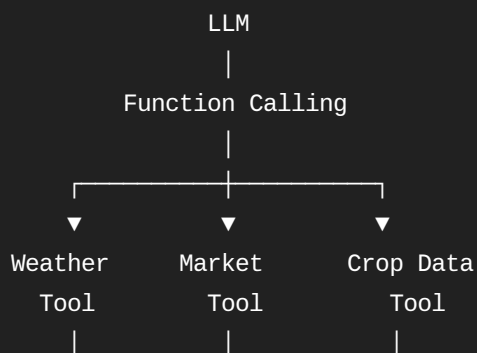
COMMUNITY_ANSWERS

answer_id
post_id
user_id
answer
is_ai_generated

One Important Architecture Decision

Don't let the LLM directly call everything.

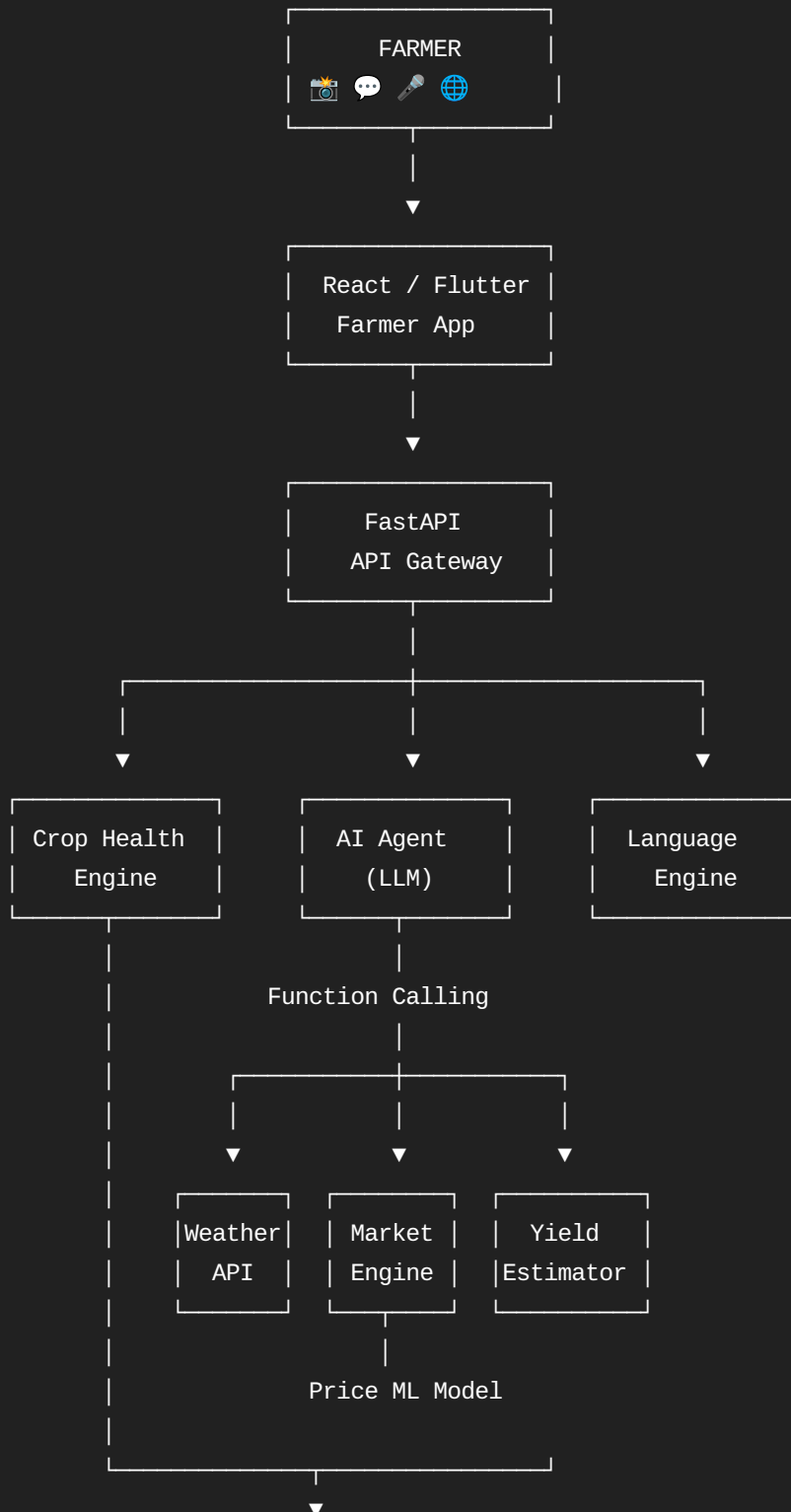
Instead:

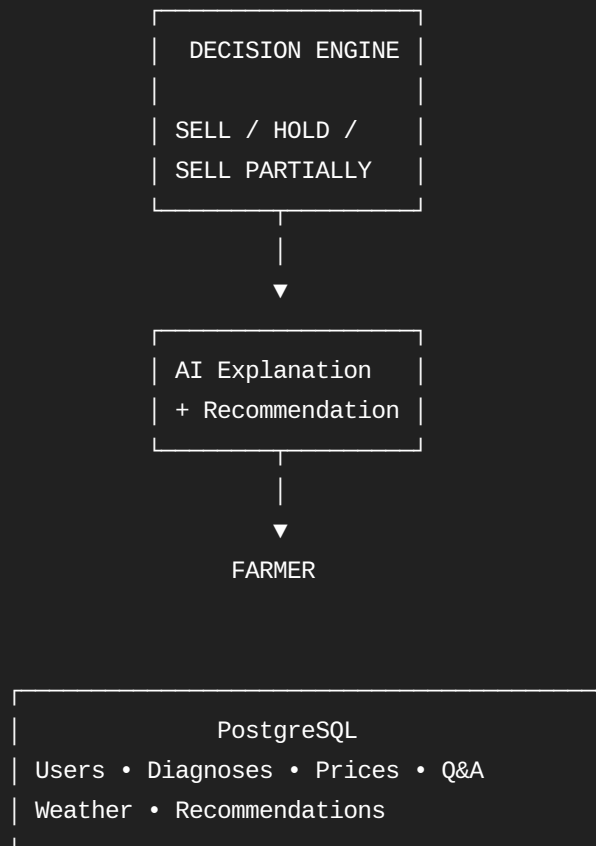


This gives you a much more credible **AI Agent architecture**.

🧩 Complete Architecture

So if you need to put **one architecture diagram in your hackathon PPT**, I'd use this:





🎯 The core principle

Models calculate → APIs provide real data → Decision Engine makes the structured decision → LLM explains it to the farmer.

That separation is important. It makes your project **technically defensible**, easier to debug, and much easier for different teammates to work on simultaneously.

And **WhatsApp stays completely outside this architecture for now.**

Later, it can simply become another frontend/channel connecting to the same FastAPI backend.